AI REPORT GENERATOR

Automated Weekly Progress Reports

Google Sheets → AI (Groq) → Email → Cron

A complete guide to building an automated AI-powered report pipeline using Python

AI Training 2026
Kigali, Rwanda
May 2026

Table of Contents

1. Project Overview

This document is a complete, step-by-step guide to building an automated AI report generator. By the end, you will have a Python script that:

- **Reads real student data** from a Google Sheets spreadsheet
- **Processes and summarises** the data (averages, at-risk students, top performers)
- **Generates a professional report** using an AI language model (Groq/Llama 3.3)
- **Sends the report as a real email** via Gmail SMTP
- **Runs automatically every Monday** using a Linux cron job

This project demonstrates how AI workflow automation works in practice. It mirrors the logic of visual workflow tools like n8n, but implemented entirely in Python so you understand every moving part.

2. Architecture and Workflow

The project follows a five-stage pipeline. Each stage maps to a node in an n8n-style workflow:

Stage	Component	Description
1. Trigger	Cron Job	Runs the script every Monday at 8:00 AM automatically
2. Data Source	Google Sheets API	Reads student names, attendance, and scores from a live spreadsheet
3. Transform	Python Function	Calculates averages, identifies at-risk and top students
4. AI Generation	Groq API (Llama 3.3)	Sends a prompt with the summary data and receives a written report
5. Output	Gmail SMTP	Emails the AI-generated report to the teacher

Data flow: Google Sheet → Python processing → Groq AI → Gmail → Teacher's inbox

3. Prerequisites

Before you begin, make sure you have the following:

Requirement	Details
Python 3.8+	Check with: python --version
Google Account	For Google Sheets and Gmail
Groq Account	Free at console.groq.com (no credit card needed)
Gmail App Password	Requires 2-Step Verification enabled on your Google account
Linux/WSL Terminal	For running scripts and setting up cron jobs

Install the required Python libraries:

```
python -m pip install gspread
python -m pip install groq
```

4. Step 1: Set Up Google Sheets as Data Source

Create a Google Sheets spreadsheet that will hold your student data. This is the live data source that the script reads from every time it runs.

Create the Spreadsheet

1. Go to sheets.google.com and create a new spreadsheet
2. Name it **Student Progress 2026**
3. Add three column headers in Row 1:

Name	Attendance	Score
Amina Uwase	TRUE	87

Patrick Mugisha	TRUE	54
Grace Ishimwe	FALSE	72
Eric Niyonzima	TRUE	91
Diane Mukamana	TRUE	48

Important: The column headers must be exactly Name, Attendance, and Score (case-sensitive). The Attendance column uses TRUE or FALSE values.

5. Step 2: Create a Google Service Account

A service account lets your Python script access Google Sheets without needing to log in manually. Think of it as a robot Google account that your script uses.

Create the Service Account

1. Go to console.cloud.google.com
2. Create a new project (or select an existing one)
3. Go to **APIs & Services → Library**
4. Search for **Google Sheets API** → click **Enable**
5. Go to **APIs & Services → Credentials**
6. Click **Create Credentials → Service Account**
7. Name it **report-bot** and finish the wizard
8. Click on the service account you just created
9. Go to **Keys → Add Key → Create New Key → JSON**
10. A JSON file downloads — this is your credentials file

Save the Credentials File

Move the downloaded JSON file to your home directory and rename it:

```
cp ~/Downloads/your-project-name.json ~/credentials.json
```

Share the Sheet with the Bot

1. Open `credentials.json` and find the **client_email** field
2. Go to your Google Sheet and click Share
3. Paste the client_email address and give it Viewer access
4. Click Send

Without this step, the script cannot access your spreadsheet and will throw a permission error.

6. Step 3: Set Up the AI Provider (Groq)

Groq provides free access to powerful AI language models. We use Llama 3.3 70B, which runs extremely fast on Groq's hardware.

Get Your API Key

1. Go to console.groq.com and sign up (free, no credit card)
2. Go to **API Keys** → **Create New Key**
3. Copy the key

Set the Environment Variable

In your terminal, run:

```
export GROQ_API_KEY="gsk_your_key_here"
```

Why Groq? It is completely free, requires no credit card, works from any country, and Llama 3.3 70B produces high-quality text. You can swap to other providers (OpenAI, Anthropic, Google) by changing one function.

7. Step 4: Set Up Email Sending (Gmail SMTP)

To send real emails from Python, you need a Gmail App Password. This is different from your normal Gmail password — it is a special 16-character code that lets scripts send email on your behalf.

Enable 2-Step Verification

1. Go to myaccount.google.com
2. Click **Security** → **2-Step Verification**
3. Follow the prompts to set it up with your phone number

Create the App Password

4. Go to myaccount.google.com/apppasswords
5. Type **report-bot** as the app name
6. Click Create
7. Copy the 16-character password (remove spaces)

Set the Environment Variables

```
export GMAIL_ADDRESS="your.email@gmail.com"
export GMAIL_APP_PASSWORD="abcdefghijklmnop"
```

8. Step 5: The Complete Python Script

Create a file called report_generator.py with the following code. This is the entire project — everything in one file.

File: report_generator.py

```
from groq import Groq
import gspread
import smtplib
```

```
import os
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from datetime import datetime

# ── 1. GOOGLE SHEETS NODE (reads real data) ──
def read_student_data():
    gc = gspread.service_account(
        filename=os.path.expanduser("~/credentials.json")
    )
    sheet = gc.open_by_url(
        "https://docs.google.com/spreadsheets/d/YOUR_SHEET_ID"
    ).sheet1

    rows = sheet.get_all_records()
    students = []
    for r in rows:
        students.append({
            "name": str(r.get("Name", "")),
            "attendance": str(r.get("Attendance", "")).upper() == "TRUE",
            "score": int(r.get("Score", 0)),
        })
    return students

# ── 2. CODE NODE (formats and summarises) ──
def process_student_data(students):
    total_scores = [s["score"] for s in students]
    avg_score = round(sum(total_scores) / len(total_scores), 1)
    at_risk = [s["name"] for s in students if s["score"] < 60]
    top_performers = [s["name"] for s in students if s["score"] >= 85]
    absent = [s["name"] for s in students if not s["attendance"]]
    attendance_rate = round(
        sum(1 for s in students if s["attendance"]) / len(students) * 100
    )
    return {
        "avg_score": avg_score, "at_risk": at_risk,
        "top_performers": top_performers,
        "absent_students": absent,
        "attendance_rate": attendance_rate,
        "total_students": len(students),
        "week": "Week 1",
        "course": "AI Training 2026",
```

```
    }

# ── 3. AI NODE (Groq API) ──
def generate_report(summary):
    client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
    prompt = f"""You are an assistant helping a teacher...
    Class average: {summary['avg_score']}%
    At-risk: {', '.join(summary['at_risk'])}
    Write a short weekly progress report email."""
    response = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[{"role": "user", "content": prompt}],
        max_tokens=512,
    )
    return response.choices[0].message.content

# ── 4. SEND EMAIL NODE ──
def send_email(summary, report_text, recipient):
    sender = os.environ.get("GMAIL_ADDRESS")
    password = os.environ.get("GMAIL_APP_PASSWORD")
    msg = MIMEMultipart()
    msg["From"] = sender
    msg["To"] = recipient
    msg["Subject"] = f"{summary['week']} Progress Report"
    msg.attach(MIMEText(report_text, "plain"))
    with smtplib.SMTP("smtp.gmail.com", 587) as server:
        server.starttls()
        server.login(sender, password)
        server.send_message(msg)

# ── MAIN ──
if __name__ == "__main__":
    RECIPIENT = "teacher@example.com"  # Change this
    students = read_student_data()
    summary = process_student_data(students)
    report = generate_report(summary)
    send_email(summary, report, RECIPIENT)
    print("Workflow completed successfully")
```

9. Step 6: Running the Workflow

Once everything is set up, run the script from your terminal:

```
python report_generator.py
```

You should see output like this:

```
🔄 Running workflow...
  [1/4] Reading student data from Google Sheets...
        Found 5 students
  [2/4] Processing data...
        Avg score: 70.4%, at-risk: 2
  [3/4] Generating report with AI...
  [4/4] Sending email...
  ☑ Email sent to teacher@example.com
  ☑ Workflow completed successfully
```

Check your email inbox — the AI-generated report should arrive within seconds.

10. Step 7: Automating with Cron

A cron job makes your script run automatically on a schedule, without you needing to type anything. This is the equivalent of the Schedule Trigger node in n8n.

Create a Launcher Script

First, create a shell script that sets the environment variables and runs the Python script:

```
#!/bin/bash
# File: ~/run_report.sh

export GROQ_API_KEY="gsk_your_key_here"
export GMAIL_ADDRESS="your.email@gmail.com"
export GMAIL_APP_PASSWORD="your_16_char_password"

cd /home/yourusername
python report_generator.py >> ~/report_log.txt 2>&1
```

Make it executable:

```
chmod +x ~/run_report.sh
```

Add the Cron Job

Open the cron editor:

```
crontab -e
```

Add this line at the bottom:

```
0 8 * * 1 /home/yourusername/run_report.sh
```

This cron expression means:

Field	Value	Meaning
Minute	0	At minute 0
Hour	8	At 8:00 AM
Day of Month	*	Every day of the month
Month	*	Every month
Day of Week	1	Monday only

Verify your cron job is saved:

```
crontab -l
```

11. Script Breakdown: What Each Part Does

This section explains every function in the script so you understand exactly what is happening at each stage.

Function 1: read_student_data()

Purpose: Connect to Google Sheets and download the student data.

n8n equivalent: Google Sheets node

This function uses the gspread library to authenticate with a service account, open a specific spreadsheet by URL, and read all rows. It returns a list of Python dictionaries, each containing a student's name, attendance status, and score.

Key concept: The service account credentials file (credentials.json) acts as the login. The spreadsheet must be shared with the service account's email address.

Function 2: process_student_data()

Purpose: Clean, calculate, and summarise the raw data.

n8n equivalent: Code node

This is where the data transformation happens. The function calculates the class average score, identifies students scoring below 60% (at-risk), identifies students scoring above 85% (top performers), counts absent students, and calculates the attendance rate. It returns a summary dictionary that the AI will use to generate the report.

Function 3: generate_report()

Purpose: Send the summary data to an AI model and get back a written report.

n8n equivalent: AI node

This function constructs a detailed prompt that tells the AI exactly what to write. It includes the summary data (averages, names, percentages) and specific instructions about tone, length, and structure. The AI returns a complete, professional email body. We use Groq's Llama 3.3 70B model, which is free and fast.

Function 4: send_email()

Purpose: Send the AI-generated report as a real email.

n8n equivalent: Send Email node

This function uses Python's built-in smtplib to connect to Gmail's SMTP server on port 587, authenticate with your app password, and send the email. It uses MIMEMultipart to construct a proper email with From, To, Subject, and Body fields.

12. How to Customize the Project

Here are the most common modifications you might want to make, with exact instructions for each.

Change the Recipient Email

Open `report_generator.py` and find this line near the bottom:

```
RECIPIENT = "teacher@example.com"
```

Change it to any email address you want to receive the report.

Change the Google Sheet

In the `read_student_data()` function, replace the spreadsheet URL:

```
sheet = gc.open_by_url(
    "https://docs.google.com/spreadsheets/d/NEW_SHEET_ID"
).sheet1
```

Make sure the new sheet is shared with your service account email and has the same column headers (Name, Attendance, Score).

Change the AI Model

In the `generate_report()` function, change the model name:

```
# Current:
model="llama-3.3-70b-versatile"

# Other free Groq models:
model="llama-3.1-8b-instant"    # Faster, smaller
model="mixtral-8x7b-32768"      # Good alternative
```

Change the At-Risk Threshold

In `process_student_data()`, change the number 60 to your preferred cutoff:

```
# Currently flags students below 60%
at_risk = [s["name"] for s in students if s["score"] < 60]

# To flag students below 70% instead:
at_risk = [s["name"] for s in students if s["score"] < 70]
```

Change the Schedule

Edit your cron job with crontab -e and change the schedule:

```
# Every Monday at 8 AM (current):
0 8 * * 1 /home/yourusername/run_report.sh

# Every weekday at 7:30 AM:
30 7 * * 1-5 /home/yourusername/run_report.sh

# Every Friday at 5 PM:
0 17 * * 5 /home/yourusername/run_report.sh

# First day of every month at 9 AM:
0 9 1 * * /home/yourusername/run_report.sh
```

Send to Multiple Recipients

Change the RECIPIENT variable to a list and loop through it:

```
RECIPIENTS = [
    "teacher1@example.com",
    "teacher2@example.com",
    "principal@example.com",
]

for email in RECIPIENTS:
    send_email(summary, report, email)
    print(f"  Sent to {email}")
```

Change the Report Style

Edit the prompt inside `generate_report()`. The AI follows whatever instructions you give it:

```
# For a formal report:
"Write a formal academic progress report..."

# For a brief summary:
"Write a 3-sentence summary of this week..."

# For a report in French:
"Write the report in French..."

# For a report with action items:
"End with a numbered list of action items..."
```

13. Troubleshooting Common Errors

Error	Cause and Fix
ModuleNotFoundError: No module named 'groq'	Run: python -m pip install groq (use the same python that runs your script)
gspread.exceptions.SpreadsheetNotFound	The spreadsheet URL is wrong, or you did not share the sheet with the service account email
google.auth.exceptions.DefaultCredentialsError	The credentials.json file is missing or the path is wrong. Check ~/credentials.json exists
SMTP AuthenticationError	Wrong Gmail app password, or 2-Step Verification is not enabled on your Google account
429 Rate Limit / ResourceExhausted	You have hit the API rate limit. Wait a minute and try again, or the free tier is not available in your region
Permission denied (cron)	Make sure run_report.sh is executable: chmod +x ~/run_report.sh

14. Summary

You have built a fully automated AI report generation pipeline that:

Component	Technology	What It Does
Data Source	Google Sheets + gspread	Reads live student data from a real spreadsheet
Processing	Python functions	Calculates averages, flags at-risk students, identifies top performers
AI Generation	Groq API + Llama 3.3	Turns raw numbers into a professional, readable report
Email Delivery	Gmail SMTP	Sends the report directly to the teacher’s inbox
Scheduling	Linux Cron	Runs the entire pipeline every Monday at 8 AM, no human needed

The entire project runs in one Python file with four functions. Each function maps directly to a node in visual workflow tools like n8n. To move this to n8n, you would simply replace each function with the corresponding visual node — the logic remains identical.